Timers, Timing and PWM

Introduction to ARM Cortex-M Microprocessors — Chapter 10

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Matters
2. Why Software Delays Fail
3. Timer Architecture
4. Alternate Functions
5. PWM
6. Addendum

Why This Chapter Matters

`delay_ms()` has three problems. Its timing changes with the optimizer and the clock. It uses the CPU while it waits. It gives no accurate time base. A hardware timer solves all three. It counts on its own, so it can **schedule** an event, **measure** an event, or **generate** a timed output such as PWM.

After this chapter, you can:

1. Explain how prescaler and reload set a timer's period.
2. Calculate prescaler and reload values.
3. Generate periodic update interrupts.
4. Keep a millisecond time base with SysTick.
5. Choose the alternate function that routes a pin to a timer channel.
6. Calculate PWM frequency and duty cycle.
7. Apply PWM to LED brightness and motor speed.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 **Why Software Delays Fail**

3 Timer Architecture

4 Alternate Functions

5 PWM

6 Addendum

Three Things Wrong With `delay_ms()`

```
void delay_ms(int n) {
    for (volatile int i = 0;
    i < n * 4000; i++) ;
}
```

1. Its timing changes with the compiler optimisation level and the clock frequency
2. It consumes 100% of the CPU while waiting
3. It cannot be interrupted accurately — any ISR that fires makes the delay longer

A timer is a counter in hardware

It runs independently of the processor, at a rate you configure. It does not care what the CPU is doing, what the optimiser did, or how many interrupts fired. That is what makes it a clock rather than a guess.

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

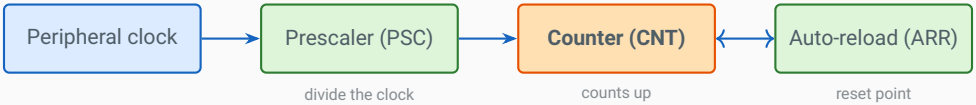
3 **Timer Architecture**

4 Alternate Functions

5 PWM

6 Addendum

Prescaler and Auto-Reload



$$f_{\text{event}} = \frac{f_{\text{CK}}}{(\text{PSC} + 1) \times (\text{ARR} + 1)}$$

The $+1$ is because a value of 0 means “divide by 1” — a counter that reloads at 0 still counts one tick.

© Muním Zabidi

A Timing Calculation

Required

A 1 Hz interrupt from a 16 MHz clock.

Choose a prescaler

PSC $=$ 15999 divides 16 MHz down to 1 kHz — a convenient 1 ms tick.

Then the reload value

ARR $=$ 999 gives 1000 ticks $=$ 1 s.

Check: $\dfrac{16{,}000{,}000}{16000 \times 1000} = 1\,\text{Hz}.$

```
TIM2->PSC = 15999;
TIM2->ARR = 999;
```

Why choose PSC first

Both registers are 16-bit, so neither can exceed 65535. Pick the prescaler to give a convenient tick — 1 ms or 1 µs — then the reload value is just the number of ticks you want.

A Periodic Timer Interrupt

```
RCC->APB1ENR |= (1U << 0);      // TIM2 clock
TIM2->PSC = 15999;              // 16 MHz -> 1 kHz
TIM2->ARR = 999;                // 1000 ticks -> 1 Hz
TIM2->DIER |= (1U << 0);        // update interrupt enable
TIM2->CR1  |= (1U << 0);        // start counting
NVIC_EnableIRQ(TIM2_IRQn);

void TIM2_IRQHandler(void)
{
    if (TIM2->SR & (1U << 0)) { // update flag?
        TIM2->SR &= ~(1U << 0); // clear it
        GPIOA->ODR ^= (1U << 5);
    }
}
```

The CPU is now free between interrupts — it can sleep, or do real work, and the LED still blinks at exactly 1 Hz.

- A 24-bit down-counter built into the core, not the vendor's peripherals.
- Present on every Cortex-M, at the same address — so code using it is portable.
- Normally configured for a 1 ms tick.

```
SysTick_Config(SystemCoreClock / 1000);
```

```
volatile uint32_t ticks = 0;

void SysTick_Handler(void)
{
    ticks++;
}

uint32_t millis(void)
{
    return ticks;
}
```

This is what replaces `delay_ms()`

Instead of *waiting*, record when something last happened and check whether enough time has passed. That is the non-blocking pattern Chapter 14 builds a whole system from.

© Muním Zabidi

The Bad `delay_ms()` — For Comparison

```c
void delay_ms_bad(int n)
{
    for (volatile int i = 0; i < n * 4000; i++)
    ;
    /* Wrong in three ways: drifts with optimisation level
    and clock frequency, burns 100% CPU, and any ISR that
    fires makes it longer than requested. Do not use this
    in real designs. */
}
```

A Proper `delay_ms()` — Built on SysTick

```c
static volatile uint32_t systick_ms = 0;

void SysTick_Handler(void) { systick_ms++; }

void systick_init(uint32_t core_clock_hz)
{
    SysTick_Config(core_clock_hz / 1000U); // 1 ms tick
}

uint32_t millis(void) { return systick_ms; }

// Blocking, but exact regardless of compiler optimisation:
// it counts real elapsed ticks, not a fixed loop count.
void delay_ms(uint32_t n)
{
    uint32_t start = millis();
    while ((millis() - start) < n) {
        __asm volatile ("wfi");  // sleep between ticks
    }
}
```

An Alternative: `delay_ms()` on a General-Purpose Timer

```
void tim2_delay_init(void)
{
    RCC->APB1ENR |= (1U << 0);  // TIM2 clock enable
    TIM2->PSC = 15999;          // 16 MHz -> 1 kHz
    TIM2->ARR = 0xFFFF;         // free-running
    TIM2->CR1 |= (1U << 0);     // start counting
}

void delay_ms_tim2(uint32_t n)
{
    TIM2->CNT = 0;
    while (TIM2->CNT < n)
    ;
}
```

Useful when SysTick is already claimed by an RTOS, and the mechanism is the same prescaler/auto-reload arithmetic from earlier in this chapter.

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

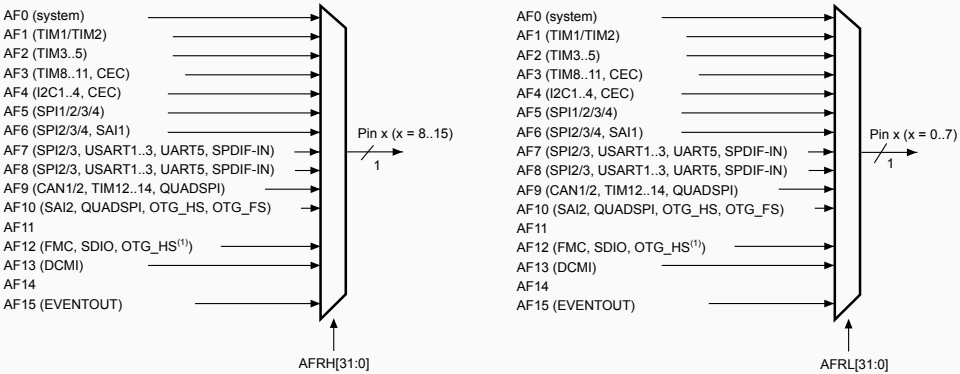
3 Timer Architecture

4 **Alternate Functions**

5 PWM

6 Addendum

Routing a Pin to a Peripheral



A pin can serve several peripherals. `MODER` set to `10` selects alternate-function mode; `AFR` then chooses which of up to sixteen functions.

Selecting the Function Number

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Four bits per pin, so `AFR[0]` covers pins 0–7 and `AFR[1]` covers pins 8–15.

For pin $n \geq 8$ the shift is $(n - 8) \times 4$.

```
// PA5 -> TIM2_CH1 is AF1
GPIOA->MODER &= ~(3U << 10);
GPIOA->MODER |=  (2U << 10);

GPIOA->AFR[0] |= (1U << (5*4));
```

The function numbers are in the datasheet, not the reference manual — a distinction that costs people an afternoon.

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

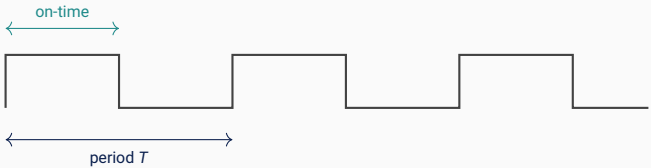
3 Timer Architecture

4 Alternate Functions

5 **PWM**

6 Addendum

Pulse Width Modulation



$$\text{duty cycle} = \frac{\text{on-time}}{T} \qquad f_{\text{PWM}} = \frac{f_{\text{CK}}}{(\text{PSC}+1)(\text{ARR}+1)}$$

A digital pin producing an analog effect

The pin is only ever fully on or fully off — but switch it fast enough and an LED looks dimmed, a motor runs slower, a heater delivers less power. The average is what the load responds to.

Configuring PWM

```
TIM2->PSC  = 15;                // 16 MHz -> 1 MHz
TIM2->ARR  = 999;               // 1 MHz / 1000 = 1 kHz PWM
TIM2->CCR1 = 250;               // 250/1000 = 25% duty

TIM2->CCMR1 |= (6U << 4);       // PWM mode 1 on channel 1
TIM2->CCMR1 |= (1U << 3);       // preload enable
TIM2->CCER  |= (1U << 0);       // enable the output
TIM2->CR1   |= (1U << 0);       // start
```

ARR sets the frequency, CCR sets the duty

Change CCR1 at any time and the duty cycle changes on the next cycle — no glitch, because the preload register updates only at the reload point. That is what makes smooth dimming possible.

A higher PWM frequency is less visible but wastes more switching energy. For LEDs, 1 kHz is comfortably above flicker perception.

© Muním Zabidi

PWM Duty-Cycle Helper

```c
void pwm_init_tim2_ch1(uint32_t psc, uint32_t arr)
{
    RCC->APB1ENR |= (1U << 0);
    TIM2->PSC = psc;
    TIM2->ARR = arr;
    TIM2->CCR1 = 0;                // start at 0% duty
    TIM2->CCMR1 |= (6U << 4);      // PWM mode 1, channel 1
    TIM2->CCMR1 |= (1U << 3);      // preload enable
    TIM2->CCER  |= (1U << 0);      // enable output
    TIM2->CR1   |= (1U << 0);      // start
}

// percent: 0-100
void pwm_set_duty_percent(uint8_t percent)
{
    if (percent > 100) percent = 100;
    uint32_t arr = TIM2->ARR;
    TIM2->CCR1 = ((uint32_t)percent * (arr + 1)) / 100U;
}
```

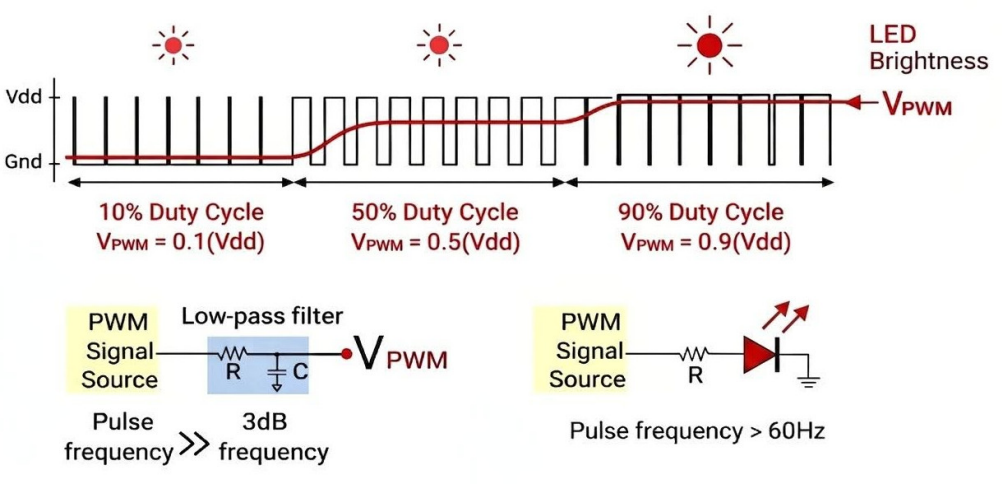
Example: A Breathing LED, Non-Blocking

```
void led_breathe_demo(void)
{
    static periodic_task_t step = { .last_ms = 0, .period_ms = 20 };
    static int duty = 0;
    static int dir  = 1;

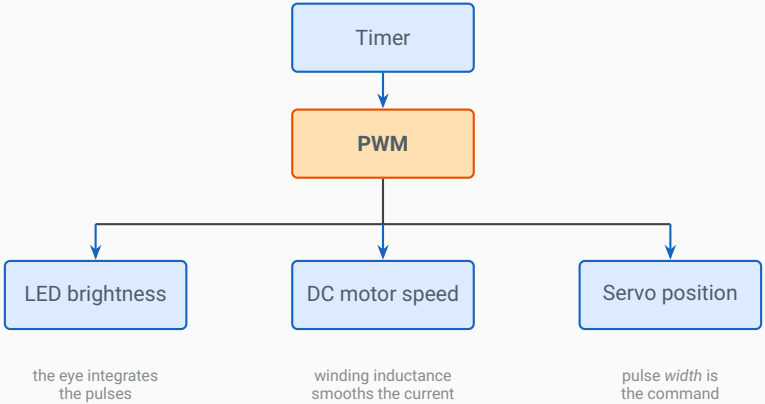
    if (task_due(&step)) {
        duty += dir;
        if (duty >= 100 || duty <= 0) dir = -dir;
        pwm_set_duty_percent((uint8_t)duty);
    }
}
```

Every 20 ms the duty cycle steps by one percentage point up or down, driven entirely by `task_due()` — no blocking call anywhere in the loop.

LED Brightness Control with PWM



One PWM Output, Three Applications



LED and motor rely on physical averaging. A servo is different: its control electronics measure the pulse width and use it to set a position.

© Muním Zabidi

PWM Output at Five Levels

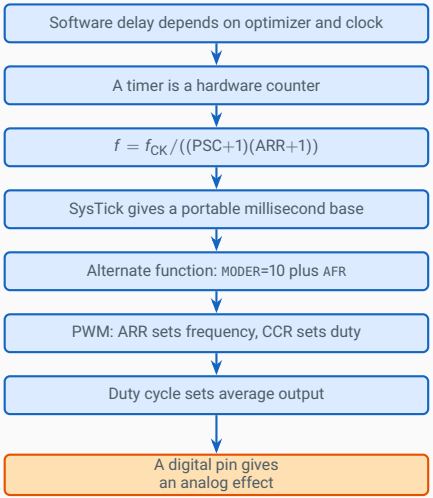
Level	Representation
Physical hardware	Counter, comparator and output flip-flop driving the pad
Register	`TIM2_CCR1` compared against `TIM2_CNT`
C	`TIM2->CCR1 = 500;`
Assembly	`MOV R0, #500` / `STR R0, [R1, #0x34]`
Observable	A 1 kHz square wave at 50% duty on pin PA5

So far

`ARR` determines the PWM period. `CCRx` determines how long the output stays high. The same two registers used earlier to schedule an interrupt now generate a waveform instead.

© Muním Zabidi

Summary



Terminology covered

- Prescaler (PSC), reload (ARR)
- SysTick
- Alternate function, `AFR`
- PWM, duty cycle, `CCR`

Next: Chapter 11

ADC and Analog Interfacing

We made an analog effect. Next, we measure a real one.

© Munim Zabidi

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

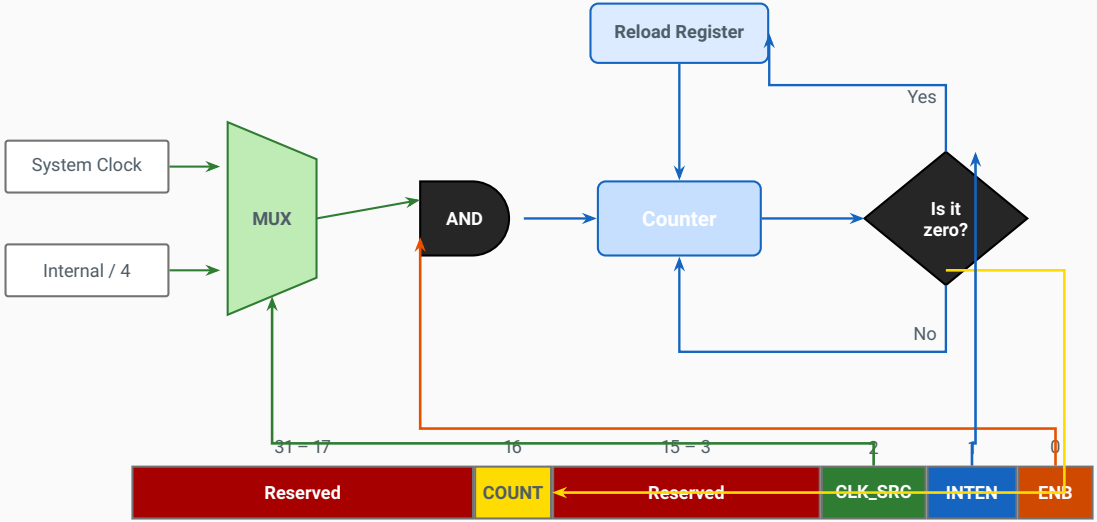
3 Timer Architecture

4 Alternate Functions

5 PWM

6 **Addendum**

SysTick, from Bits to Blocks

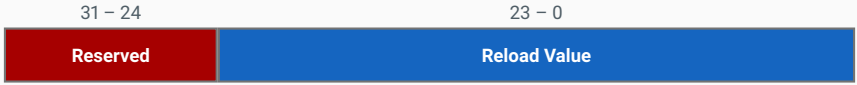


SysTick Control and Status Register

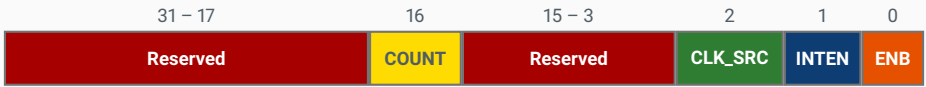
SysTick's Three Registers, Bit by Bit



SysTick Current Value Register (`VAL`)



SysTick Reload Register (`LOAD`)



SysTick Control and Status Register (`CTRL`)

Three 32-bit registers. In each one the top byte is Reserved; the `VAL`/`LOAD` halves carry the 24-bit count, and `CTRL` carries the four control bits discussed on the previous slide. CMSIS names: `SysTick->VAL`, `SysTick->LOAD`, `SysTick->CTRL`.

Register map adapted and redrawn from microcontrollerslab.com, "ARM Cortex-M4 SysTick timer with programming example".

TI, ARM, and STMicro: Same SysTick, Three Register Names

SysTick is a Cortex-M core peripheral — same registers, same bits on every vendor's chip. Only the *name* the vendor's header file uses changes.

Bit	Function	TI (TM4C)	ARM / CMSIS
0	Enable the counter	`ENB`	`ENABLE`
1	Enable the SysTick interrupt	`INTEN`	`TICKINT`
2	Clock source (1=core, 0=slow)	`CLK_SRC`	`CLKSOURCE`
16	Set when counter wraps to zero	`COUNT`	`COUNTFLAG`

One implementation choice to watch

The slow clock source is implementation-defined by the Cortex-M architecture. TI's TM4C uses the system clock divided by 4; STMicro's STM32 uses HCLK divided by 8; most other Cortex-M vendors pick 8 as well.

© Muním Zabidi

Reading the Diagram from a Program's Point of View

Write to CTRL, left to right

- `CLK_SRC` picks the clock input to the MUX.
- `INTEN` arms the interrupt path.
- `ENB` switches on the AND gate, so the clock reaches the counter.

Three writes, in that order, is what `SysTick_Config()` does under the hood.

Read COUNT, from the top

- Counter decrements each clock.
- At zero: Yes-branch reloads from the Reload Register, and sets `COUNT`.
- Reading `COUNT` (or writing `VAL`) clears it.

The ARM TRM says this bit is sticky-until-read: do not use it for timing; use the interrupt instead.

Why this picture is worth the detour

Chapter 10 showed the SysTick API. This diagram shows the hardware behind it, so an unfamiliar vendor's SysTick code is still readable: find `ENB`, `INTEN`, `CLK_SRC`, and the reload value, and you can predict the tick rate.

© Muním Zabidi

Driving the Bits Directly

No CMSIS helper. Write the four CTRL bits by hand, in the order the diagram shows them — clock source, interrupt enable, counter enable — after loading `LOAD` and clearing `VAL`.

```c
#include "stm32f4xx.h"                /* STM32F446RE, Cortex-M4 */

#define SYST_CTRL_ENABLE     (1U << 0) /* bit 0  -- ENB */
#define SYST_CTRL_TICKINT    (1U << 1) /* bit 1  -- INTEN */
#define SYST_CTRL_CLKSOURCE  (1U << 2) /* bit 2  -- CLK_SRC (1 = processor clock) */

void systick_init_1ms(void) {
    SysTick->LOAD = (SystemCoreClock / 1000U) - 1U;   /* 24-bit reload */
    SysTick->VAL  = 0;                                /* clears count + COUNTFLAG */
    SysTick->CTRL = SYST_CTRL_CLKSOURCE               /* use processor clock */
                  | SYST_CTRL_TICKINT                 /* raise an interrupt  */
                  | SYST_CTRL_ENABLE;                 /* start the counter   */
}
```

The two things CMSIS does for you

`SysTick_Config()` wraps the same four writes plus an `NVIC_SetPriority(SysTick_IRQn, ...)` call, and returns `-1` if the reload value overflows 24 bits. Prefer the wrapper in portable code; use the raw form when you need a specific priority or non-default clock source.

Handler and Millisecond Delay

```c
static volatile uint32_t msTicks = 0;    /* ISR-shared, must be volatile */

void SysTick_Handler(void) {             /* fires every 1 ms */
    msTicks++;
}

void delay(uint32_t ms) {
    uint32_t start = msTicks;
    while ((msTicks - start) < ms) {
        __WFI();                         /* sleep until the next tick */
    }
}
```

Why unsigned subtraction

`msTicks - start` stays correct even after `msTicks` wraps at 2^{32}, because unsigned wrap-around subtracts cleanly. `while (msTicks < start + ms)` would fail near the wrap.

Why `__WFI()`

"Wait For Interrupt" sleeps the core until the next SysTick fires. On a battery-powered board this is the difference between a few µA and tens of mA.

Handler and `delay()` pattern adapted from Rohit Nimkar, `csrohit/bluepill-systick` (BSD 3-Clause), with `__WFI()` added. Full source: `github.com/csrohit/bluepill-systick`.

Portable: SysTick. Not Portable: Everything Around It.

Same CMSIS source runs on both boards. The lines that change are the ones that touch the vendor's silicon.

	Nucleo-F446RE	Blue Pill (STM32F103)
SysTick code — compiles unchanged		
`SysTick_Config(SystemCoreClock/1000)`	16 000 (after reset)	8 000 (after reset)
`SysTick->CTRL/LOAD/VAL`, bit names	identical	identical
`SysTick_Handler`, `__WFI()`	identical	identical
Lines that must change		
Core	Cortex-M4 (FPU, DSP)	Cortex-M3 (no FPU, no DSP)
Max clock	180 MHz	72 MHz
CMSIS header	`stm32f4xx.h`	`stm32f1xx.h`
LED pin / polarity	PA5, active-high	PC13, active-low (hw)
Port clock enable	`RCC->AHB1ENR` bit 0 (GPIOA)	`RCC->APB2ENR` bit 4 (GPIOC)
Pin mode register	`MODER` (2 bits/pin)	`CRH/CRL` (4 bits/pin)
Programmer	on-board ST-Link v2.1	external ST-Link v2 (SWD)

The pattern

SysTick is in the Arm core, so it is portable across every vendor's Cortex-M. GPIO, clocks and the LED are in the vendor's silicon, so they change on every chip.

© Muním Zabidi